SRS(Software Requirement Specifications)

QR Cafe Firestore App

By: [Student Name]
Adm No.: [Admission No.]

1. Introduction

1.1 Purpose

The purpose of this project is to develop a web-based QR cafe ordering platform that helps cafe owners and customers manage table-based food ordering efficiently. Customers can scan a QR code placed on a table, browse the menu, place orders, and call a waiter without waiting for manual service. Cafe owners and staff can manage menu items, tables, orders, themes, cashier workflows, and analytics from a centralized dashboard.

1.2 Scope

The QR Cafe Firestore App is a full-stack web application that allows users to:

- Register and authenticate cafe owners securely.
- Create and manage cafe tables with QR codes.
- Allow customers to open a table-specific digital menu by scanning a QR code.
- Place food orders directly from the customer menu page.
- Call a waiter from the table interface.
- Manage menu items, table data, theme settings, cashier orders, and sales analytics.
- Store cafe data using Firebase Authentication and Cloud Firestore.

The system provides a digital ordering experience for cafes, reduces manual order handling, and improves service speed by connecting customer actions directly to cafe staff workflows.

1.3 Objectives

The main objectives of the QR Cafe Firestore App are:

- Provide a simple QR-based ordering system for cafes and restaurants.
- Enable table-wise customer ordering without requiring a mobile app installation.
- Allow cafe owners to manage menus, tables, and order status from a web dashboard.
- Support waiter call requests and active order monitoring for cashier staff.
- Use Firestore to store cafe-specific menu items, tables, orders, waiter calls, and theme data.
- Improve operational efficiency, order accuracy, and customer convenience.

2. Description

2.1 Product Perspective

The project is a web-based application developed using Node.js, Express, Firebase Authentication, Cloud Firestore, and vanilla HTML, CSS, and JavaScript. It provides public customer pages for ordering and protected owner/admin pages for cafe management.

Main Tech Stacks include:

- HTML, CSS, and vanilla JavaScript for the frontend user interface.
- Node.js and Express.js for backend server and API handling.
- Firebase Authentication for secure cafe owner sign-up and sign-in.
- Cloud Firestore for menu, table, order, waiter call, theme, and analytics data.
- Firebase Admin SDK for server-side authentication and Firestore operations.
- qrcode package for generating QR codes for cafe tables.
- Environment-based Firebase configuration for server and browser setup.

The platform provides a centralized system for cafe owners, cashier staff, and customers to manage menu browsing, table-based ordering, waiter assistance, and order tracking.

2.2 Product Functions

Main Functions:

- Cafe owner registration and authentication.
- Dashboard for high-level cafe statistics.
- Menu item creation, update, listing, and deletion.
- Table setup and QR code generation.
- Customer menu page for table-specific ordering.
- Cashier page for active orders and waiter calls.

Smart Features:

- Automatic QR code generation for each cafe table.
- Theme customization for the customer menu experience.
- Order tracking by cafe and table number.
- Waiter call management for faster service response.
- Analytics views for sales and cafe activity insights.

3. System Users

Cafe Owner/Admin - Account setup, menu management, table management, theme settings, and analytics.

Cashier/Staff - Active order monitoring, order status updates, and waiter call handling.

Customer - Scans a table QR code, browses the menu, places orders, and calls a waiter.

4. Functional Requirements

4.1 Authentication Module

- Cafe owner sign-up and sign-in.
- Secure session creation and logout.
- Route protection for dashboard, admin, cashier, tables, and analytics pages.
- Firebase-based identity verification for protected API access.

Technologies Used: Firebase Authentication, Firebase Admin SDK, Node.js, Express.js

4.2 Menu Management Module

- Add, update, view, and delete cafe menu items.
- Store menu data under the authenticated cafe record.
- Display active menu items to customers on the table menu page.
- Support item details used by ordering and cashier workflows.

Technologies Used: HTML, CSS, JavaScript, Express.js, Cloud Firestore

4.3 Table and QR Management Module

- Create and manage table records for each cafe.
- Generate QR codes linked to the customer menu route.
- Use URLs such as /menu?cafe_id=...&table=1 for table-specific access.
- Allow cafe owners to prepare QR codes for physical table placement.

Technologies Used: qrcode, Express.js, Cloud Firestore, JavaScript

4.4 Customer Ordering Module

- Open a cafe menu after scanning a table QR code.
- Browse available menu items from the selected cafe.
- Add items to an order and submit the order for the selected table.
- Store order details for staff and cashier review.

Technologies Used: HTML, CSS, JavaScript, Express.js, Cloud Firestore

4.5 Cashier and Order Monitoring Module

- Display active customer orders for the cafe.
- Allow staff to update or remove orders based on workflow needs.
- Refresh active orders periodically for near real-time monitoring.
- Support cashier staff in tracking table requests and order activity.

Technologies Used: JavaScript, Express.js, Cloud Firestore

4.6 Waiter Call Module

- Allow customers to call a waiter from the menu page.
- Store waiter call requests with cafe and table information.
- Display active waiter calls on the cashier/staff page.
- Allow staff to clear waiter requests after service.

Technologies Used: JavaScript, Express.js, Cloud Firestore

4.7 Theme Customization Module

- Allow cafe owners to customize theme settings for the customer menu.
- Store theme configuration inside the cafe settings document.
- Apply saved visual settings to the customer-facing menu page.
- Support branding and presentation for each cafe.

Technologies Used: HTML, CSS, JavaScript, Cloud Firestore

4.8 Analytics Module

- Display sales and order-related insights for cafe owners.
- Fetch cafe-specific analytics through the backend API.
- Help owners understand activity from orders and menu usage.
- Support business monitoring from the analytics page.

Technologies Used: Express.js, JavaScript, Cloud Firestore

5. Non-Functional Requirements

5.1 Performance

- Fast loading customer menu pages for QR-based ordering.
- Efficient API responses for menu, order, table, waiter, and theme requests.
- Periodic refresh support for cashier order monitoring.
- Firestore queries scoped by cafe to keep data access efficient.

5.2 Security

- Firebase Authentication for cafe owner accounts.
- Protected owner routes and session validation.
- Server-side Firebase Admin verification for trusted operations.
- Environment variables used for sensitive Firebase configuration values.

5.3 Usability

- Simple customer menu interface opened directly from a QR scan.
- Separate pages for owner, cashier, table setup, and analytics workflows.
- Clear navigation for sign-up, sign-in, dashboard, admin, cashier, tables, and menu pages.
- Theme customization support for a better cafe-specific experience.

5.4 Scalability

- Data is organized per cafe using nested Firestore collections.
- The system can support multiple cafes with separate menu, table, order, and settings data.
- Additional modules can be added through existing Express API and Firestore structure.
- Future real-time Firestore listeners can replace timer-based cashier refreshes.

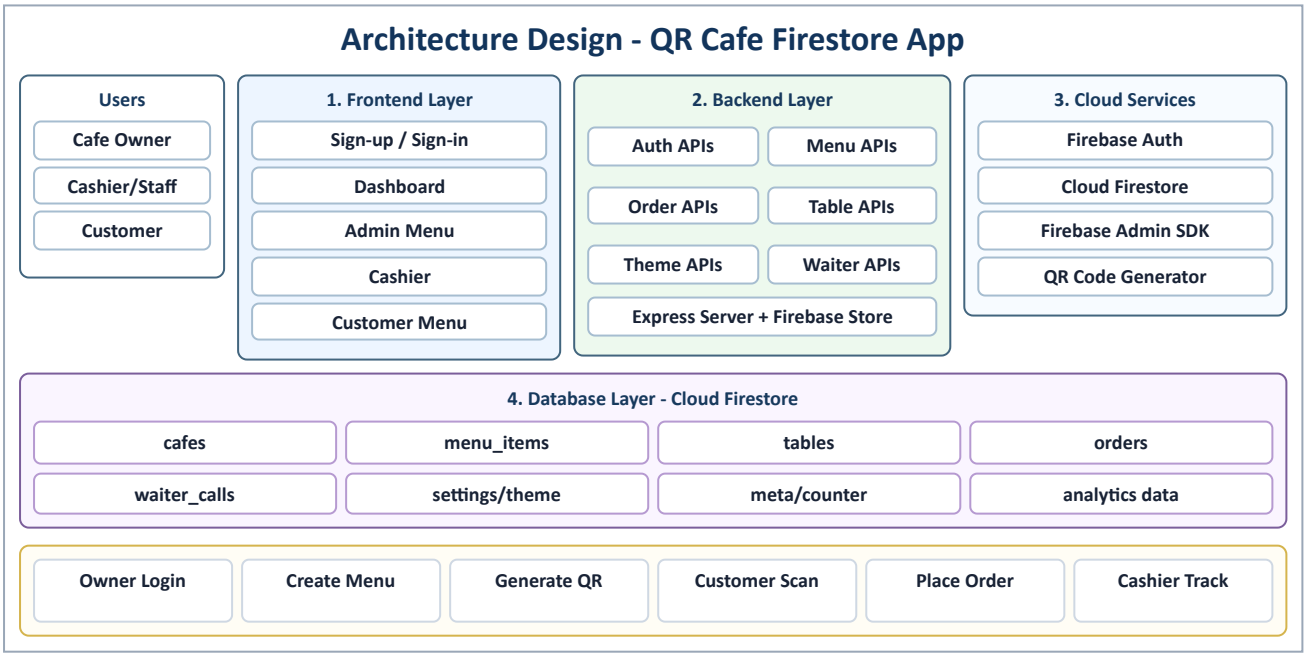
6. Software Requirements

Software/Technology	Purpose
HTML, CSS, JavaScript	Frontend pages and customer ordering interface
Node.js	Backend runtime environment
Express.js	API routing, server handling, and static page serving
Firebase Authentication	Secure cafe owner authentication
Cloud Firestore	Database for cafes, menu items, tables, orders, waiter calls, and theme settings
Firebase Admin SDK	Server-side Firebase access and authentication verification
qrcode	QR code generation for cafe tables
VS Code	Code development
Git & GitHub	Version control

7. Hardware Requirements

Hardware component	Requirement
Processor	Intel i3 or above
RAM	Minimum 4 GB
Device	Laptop, desktop, tablet, or mobile phone
Storage	Minimum 20 GB free space for development system
Internet connection	Required for Firebase and web access

8. Architecture Design



9. Database Design

Cafes Collection: cafeId, name, email, ownerId, createdAt

Menu Items Collection: itemId, cafeId, name, description, price, category, availability

Tables Collection: tableId, cafeId, tableNumber, qrCodeUrl, status

Orders Collection: orderId, cafeId, tableNumber, items, totalAmount, status, createdAt

Waiter Calls Collection: callId, cafeId, tableNumber, status, createdAt

Theme Settings Document: primaryColor, accentColor, cafeName, menuAppearance

Order Counter Metadata: counter_order, lastOrderNumber

10. Future Enhancement

- Real-time Firestore listeners for live cashier order updates.
- Online payment integration for customer checkout.
- Kitchen display system for food preparation tracking.
- Inventory management based on menu item usage.
- Customer feedback and rating system.
- Multi-branch cafe management support.
- Progressive web app installation for staff devices.

11. Conclusion

The QR Cafe Firestore App provides a digital solution for cafe ordering and table service management. The system combines QR-based customer access, Firebase Authentication, Cloud Firestore, Express APIs, menu management, order tracking, waiter calls, theme customization, and analytics to improve the overall cafe workflow. By reducing manual order collection and giving cafe staff a centralized dashboard, the application improves customer convenience, service speed, and operational efficiency.